This is CS50

CS50's Introduction to Computer Science

OpenCourseWare

Donate (https://cs50.harvard.edu/donate)

David J. Malan (https://cs.harvard.edu/malan/)
malan@harvard.edu
(https://www.facebook.com/dmalan) (https://github.com/dmalan)
(https://www.instagram.com/davidjmalan/) (https://www.linkedin.com/in/malan/)
(https://www.reddit.com/user/davidjmalan) (https://www.threads.net/@davidjmalan)
(https://twitter.com/davidjmalan)

Lecture 7

Welcome!

- In previous weeks, we introduced you to Python, a high-level programming language that utilized the same building blocks we learned in C. However, we introduced this new language not for the purpose of learning "just another language." Instead, we do so because some tools are better for some jobs and not so great for others!

- This week, we will be continuing more syntax related to Python.
- Further, we will be integrating this knowledge with data.
- Finally, we will be discussing *SQL* or *Structured Query Language*, a domain-specific way by which we can interact with and modify data.
- Overall, one of the goals of this course is to learn to program generally – not simply how to program in the languages described in this course.

Flat-File Database

- As you have likely seen before, data can often be described in patterns of columns and rows.
- Spreadsheets like those created in Microsoft Excel and Google Sheets can be outputted to a `csv` or *comma-separated values* file.
- If you look at a `csv` file, you'll notice that the file is flat in that all of our data is stored in a single table represented by a text file. We call this form of data a *flat-file database*.
- All data is stored row by row. Each column is separated by a comma or another value.
- Python comes with native support for `csv` files.
- First, download favorites.csv (https://cdn.cs50.net/2023/fall/lectures/7/src7/favorites/favorites.csv) and upload it to your file explorer inside cs50.dev (https://cs50.dev). Second, examining this data, notice that the first row is special in that it defines each column. Then, each record is stored row by row.
- In your terminal window, type `code favorites.py` and write code as follows:

```
# Prints all favorites in CSV using csv.reader

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create reader
    reader = csv.reader(file)

    # Skip header row
    next(reader)

    # Iterate over CSV file, printing each favorite
    for row in reader:
        print(row[1])
```

  Notice that the `csv` library is imported. Further, we created a `reader` that will hold the result of `csv.reader(file)`. The `csv.reader` function reads each row from the file, and in our code, we store the results in `reader`. `print(row[1])`, therefore, will print the language from the `favorites.csv` file.

- You can improve your code as follows:

```
# Stores favorite in a variable

import csv

# Open CSV file
```

```
with open("favorites.csv", "r") as file:

    # Create reader
    reader = csv.reader(file)

    # Skip header row
    next(reader)

    # Iterate over CSV file, printing each favorite
    for row in reader:
        favorite = row[1]
        print(favorite)
```

Notice that `favorite` is stored and then printed. Also, notice that we use the `next` function to skip to the next line of our reader.

- One of the disadvantages of the above approach is that we are trusting that `row[1]` is always the favorite. However, what would happen if the columns had been moved around?
- We can fix this potential issue. Python also allows you to index by the keys of a list. Modify your code as follows:

```
# Prints all favorites in CSV using csv.DictReader

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Iterate over CSV file, printing each favorite
    for row in reader:
        favorite = row["language"]
        print(favorite)
```

Notice that this example directly utilizes the `language` key in the print statement. `favorite` indexes into the `reader` dictionary of `row["language"]`.

- This could be further simplified to:

```
# Prints all favorites in CSV using csv.DictReader

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Iterate over CSV file, printing each favorite
    for row in reader:
        print(row["language"])
```

- To count the number of favorite languages expressed in the `csv` file, we can do the following:

```
# Counts favorites using variables

import csv
```

```
# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    scratch, c, python = 0, 0, 0

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        if favorite == "Scratch":
            scratch += 1
        elif favorite == "C":
            c += 1
        elif favorite == "Python":
            python += 1

# Print counts
print(f"Scratch: {scratch}")
print(f"C: {c}")
print(f"Python: {python}")
```

Notice that each language is counted using `if` statements. Further, notice the double equal `==` signs in those `if` statements.

- Python allows us to use a dictionary to count the `counts` of each language. Consider the following improvement upon our code:

```
# Counts favorites using dictionary

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    counts = {}

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        if favorite in counts:
            counts[favorite] += 1
        else:
            counts[favorite] = 1

# Print counts
for favorite in counts:
    print(f"{favorite}: {counts[favorite]}")
```

Notice that the value in `counts` with the key `favorite` is incremented when it exists already. If it does not exist, we define `counts[favorite]` and set it to 1. Further, the formatted string has been improved to present the `counts[favorite]`.

- Python also allows sorting `counts`. Improve your code as follows:

```
# Sorts favorites by key

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    counts = {}

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        if favorite in counts:
            counts[favorite] += 1
        else:
            counts[favorite] = 1

# Print counts
for favorite in sorted(counts):
    print(f"{favorite}: {counts[favorite]}")
```

Notice the `sorted(counts)` at the bottom of the code.

- If you look at the parameters for the `sorted` function in the Python documentation, you will find it has many built-in parameters. You can leverage some of these built-in parameters as follows:

```
# Sorts favorites by value using .get

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    counts = {}

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        if favorite in counts:
            counts[favorite] += 1
        else:
            counts[favorite] = 1

# Print counts
for favorite in sorted(counts, key=counts.get, reverse=True):
    print(f"{favorite}: {counts[favorite]}")
```

Notice the arguments passed to `sorted`. The `key` argument allows you to tell Python the method you wish to use to sort items. In this case `counts.get` is used to sort by the values. `reverse=True` tells `sorted` to sort from largest to smallest.

- Python has numerous libraries that we can utilize in our code. One of these libraries is `collections`, from which we can import `Counter`. `Counter` will allow you to access the counts

of each language without the headaches of all the `if` statements seen in our previous code. You can implement as follows:

```
# Sorts favorites by value using .get

import csv

from collections import Counter

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    counts = Counter()

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        counts[favorite] += 1

# Print counts
for favorite, count in counts.most_common():
    print(f"{favorite}: {count}")
```

Notice how `counts = Counter()` enables the use of this imported `Counter` class from `collections`.

- You can learn more about sorted (https://docs.python.org/3/howto/sorting.html) in the Python Documentation (https://docs.python.org/3/howto/sorting.html).

Relational Databases

- Google, X, and Meta all use relational databases to store their information at scale.
- Relational databases store data in rows and columns in structures called *tables*.
- SQL allows for four types of commands:

```
Create
Read
Update
Delete
```

- These four operations are affectionately called *CRUD*.
- We can create a database with the SQL syntax `CREATE TABLE table (column type, ...);`. But where do you run this command?
- `sqlite3` is a type of SQL database that has the core features required for this course.
- We can create a SQL database at the terminal by typing `sqlite3 favorites.db`. Upon being prompted, we will agree that we want to create `favorites.db` by pressing `y`.
- You will notice a different prompt as we are now using a program called `sqlite`.
- We can put `sqlite` into `csv` mode by typing `.mode csv`. Then, we can import our data from our `csv` file by typing `.import favorites.csv favorites`. It seems that nothing has happened!

- We can type `.schema` to see the structure of the database.
- You can read items from a table using the syntax `SELECT columns FROM table`.
- For example, you can type `SELECT * FROM favorites;` which will print every row in `favorites`.
- You can get a subset of the data using the command `SELECT language FROM favorites;`.
- SQL supports many commands to access data, including:

```
AVG
COUNT
DISTINCT
LOWER
MAX
MIN
UPPER
```

- For example, you can type `SELECT COUNT(*) FROM favorites;`. Further, you can type `SELECT DISTINCT language FROM favorites;` to get a list of the individual languages within the database. You could even type `SELECT COUNT(DISTINCT language) FROM favorites;` to get a count of those.
- SQL offers additional commands we can utilize in our queries:

```
WHERE       -- adding a Boolean expression to filter our data
LIKE        -- filtering responses more loosely
ORDER BY    -- ordering responses
LIMIT       -- limiting the number of responses
GROUP BY    -- grouping responses together
```

  Notice that we use `--` to write a comment in SQL.

SELECT

- For example, we can execute `SELECT COUNT(*) FROM favorites WHERE language = 'C';`. A count is presented.
- Further, we could type `SELECT COUNT(*) FROM favorites WHERE language = 'C' AND problem = 'Hello, World';`. Notice how the `AND` is utilized to narrow our results.
- Similarly, we could execute `SELECT language, COUNT(*) FROM favorites GROUP BY language;`. This would offer a temporary table that would show the language and count.
- We could improve this by typing `SELECT language, COUNT(*) FROM favorites GROUP BY language ORDER BY COUNT(*);`. This will order the resulting table by the `count`.
- Likewise, we could execute `SELECT COUNT(*) FROM favorites WHERE language = 'C' AND (problem = 'Hello, World' OR problem = 'Hello, It''s Me');`. Do notice that there are two `''` marks as to allow the use of single quotes in a way that does not confuse SQL.
- Further, we could execute `SELECT COUNT(*) FROM favorites WHERE language = 'C' AND problem LIKE 'Hello, %';` to find any problems that start with `Hello,` (including a space).
- We can also group the values of each language by executing `SELECT language, COUNT(*) FROM favorites GROUP BY language;`.

- We can order the output as follows: `SELECT language, COUNT(*) FROM favorites GROUP BY language ORDER BY COUNT(*) DESC;`.
- We can even create aliases, like variables in our queries: `SELECT language, COUNT(*) AS n FROM favorites GROUP BY language ORDER BY n DESC;`.
- Finally, we can limit our output to 1 or more values: `SELECT language, COUNT(*) AS n FROM favorites GROUP BY language ORDER BY n DESC LIMIT 1;`.

INSERT

- We can also `INSERT` into a SQL database utilizing the form `INSERT INTO table (column...) VALUES(value, ...);`.
- We can execute `INSERT INTO favorites (language, problem) VALUES ('SQL', 'Fiftyville');`.
- You can verify the addition of this favorite by executing `SELECT * FROM favorites;`.

DELETE

- `DELETE` allows you to delete parts of your data. For example, you could `DELETE FROM favorites WHERE Timestamp IS NULL;`. This deletes any record where the `Timestamp` is `NULL`.

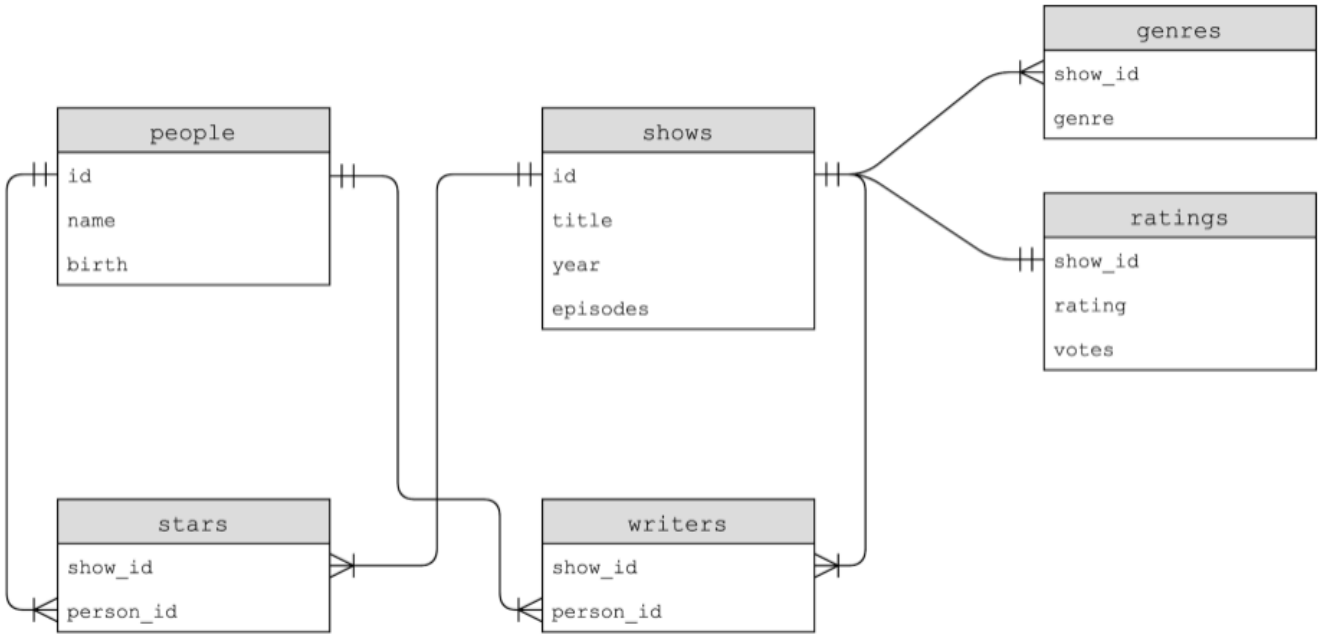
UPDATE

- We can also utilize the `UPDATE` command to update your data.
- For example, you can execute `UPDATE favorites SET language = 'SQL', problem = 'Fiftyville';`. This will result in overwriting all previous statements where C and Scratch were the favorite programming language.
- Notice that these queries have immense power. Accordingly, in the real-world setting, you should consider who has permissions to execute certain commands and if you have backups available!

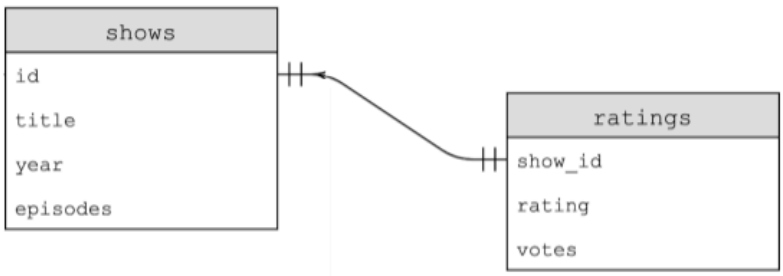
IMDb

- We can imagine a database that we might want to create to catalog various TV shows. We could create a spreadsheet with columns like `title`, `star`, `star`, `star`, `star`, and more stars. A problem with this approach is that it has a lot of wasted space. Some shows may have one star. Others may have dozens.
- We could separate our database into multiple sheets. We could have a `shows` sheet, a `stars` sheet, and a `people` sheet. On the `people` sheet, each person could have a unique `id`. On the `shows` sheet, each show could have a unique `id` too. On a third sheet called `stars` we could relate how each show has people for each show by having a `show_id` and `person_id`. While this is an improvement, this is not an ideal database.

- IMDb offers a database of people, shows, writers, stars, genres, and ratings. Each of these tables is related to one another as follows:



- After downloading `shows.db` (https://cdn.cs50.net/2024/fall/lectures/7/src7/imdb/shows.db), you can execute `sqlite3 shows.db` in your terminal window.
- Let's zero in on the relationship between two tables within the database called `shows` and `ratings`. The relationship between these two tables can be illustrated as follows:



- To illustrate the relationship between these tables, we could execute the following command: `SELECT * FROM ratings LIMIT 10;`. Examining the output, we could execute `SELECT * FROM shows LIMIT 10;`.
- Examining `shows` and `rating`, we can see these have a one-to-one relationship: One show has one rating.

- To understand the database, upon executing `.schema` you will find not only each of the tables but the individual fields inside each of these fields.
- More specifically, you could execute `.schema shows` to understand the fields inside `shows`. You can also execute `.schema ratings` to see the fields inside `ratings`.
- As you can see, `show_id` exists in all of the tables. In the `shows` table, it is simply called `id`. This common field between all the fields is called a *key*. Primary keys are used to identify a unique record in a table. *Foreign keys* are used to build relationships between tables by pointing to the primary key in another table. You can see in the schema of `ratings` that `show_id` is a foreign key that references `id` in `shows`.
- By storing data in a relational database, as above, data can be more efficiently stored.
- In *sqlite*, we have five data types, including:

```
BLOB        -- binary large objects that are groups of ones and zeros
INTEGER     -- an integer
NUMERIC     -- for numbers that are formatted specially like dates
REAL        -- like a float
TEXT        -- for strings and the like
```

- Additionally, columns can be set to add special constraints:

```
NOT NULL
UNIQUE
```

- We can further play with this data to understand these relationships. Execute `SELECT * FROM ratings;`. There are a lot of ratings!
- We can further limit this data down by executing `SELECT show_id FROM ratings WHERE rating >= 6.0 LIMIT 10;`. From this query, you can see that there are 10 shows presented. However, we don't know what show each `show_id` represents.
- You can discover what shows these are by executing `SELECT * FROM shows WHERE id = 626124;`
- We can further our query to be more efficient by executing:

```
SELECT title
FROM shows
WHERE id IN (
    SELECT show_id
    FROM ratings
    WHERE rating >= 6.0
    LIMIT 10
)
```

Notice that this query nests together two queries. An inner query is used by an outer query.

`JOIN`s

- We are pulling data from `shows` and `ratings`. Notice how both `shows` and `ratings` have an `id` in common.
- How could we combine tables temporarily? Tables could be joined together using the `JOIN` command.
- Execute the following command:

```
SELECT * FROM shows
  JOIN ratings on shows.id = ratings.show_id
  WHERE rating >= 6.0
  LIMIT 10;
```

Notice this results in a wider table than we have previously seen.

- Where the previous queries have illustrated the *one-to-one* relationship between these keys, let's examine some *one-to-many* relationships. Focusing on the `genres` table, execute the following:

```
SELECT * FROM genres
LIMIT 10;
```

Notice how this provides us a sense of the raw data. You might notice that one show has three values. This is a one-to-many relationship.

- We can learn more about the `genres` table by typing `.schema genres`.
- Execute the following command to learn more about the various comedies in the database:

```
SELECT title FROM shows
WHERE id IN (
  SELECT show_id FROM genres
  WHERE genre = 'Comedy'
  LIMIT 10
);
```

Notice how this produces a list of comedies, including *Catweazle*.

- To learn more about Catweazle, by joining various tables through a join:

```
SELECT * FROM shows
JOIN genres
ON shows.id = genres.show_id
WHERE id = 63881;
```

Notice that this results in a temporary table. It is fine to have a duplicate table.

- In contrast to one-to-one and one-to-many relationships, there may be *many-to-many* relationships.
- We can learn more about the show *The Office* and the actors in that show by executing the following command:

```
SELECT name FROM people WHERE id IN
    (SELECT person_id FROM stars WHERE show_id =
        (SELECT id FROM shows WHERE title = 'The Office' AND year = 2005));
```

Notice that this results in a table that includes the names of various stars through nested queries.

- We find all the shows in which Steve Carell starred:

```
SELECT title FROM shows WHERE id IN
    (SELECT show_id FROM stars WHERE person_id =
        (SELECT id FROM people WHERE name = 'Steve Carell'));
```

This results in a list of titles of shows wherein Steve Carell starred.

- This could also be expressed in this way:

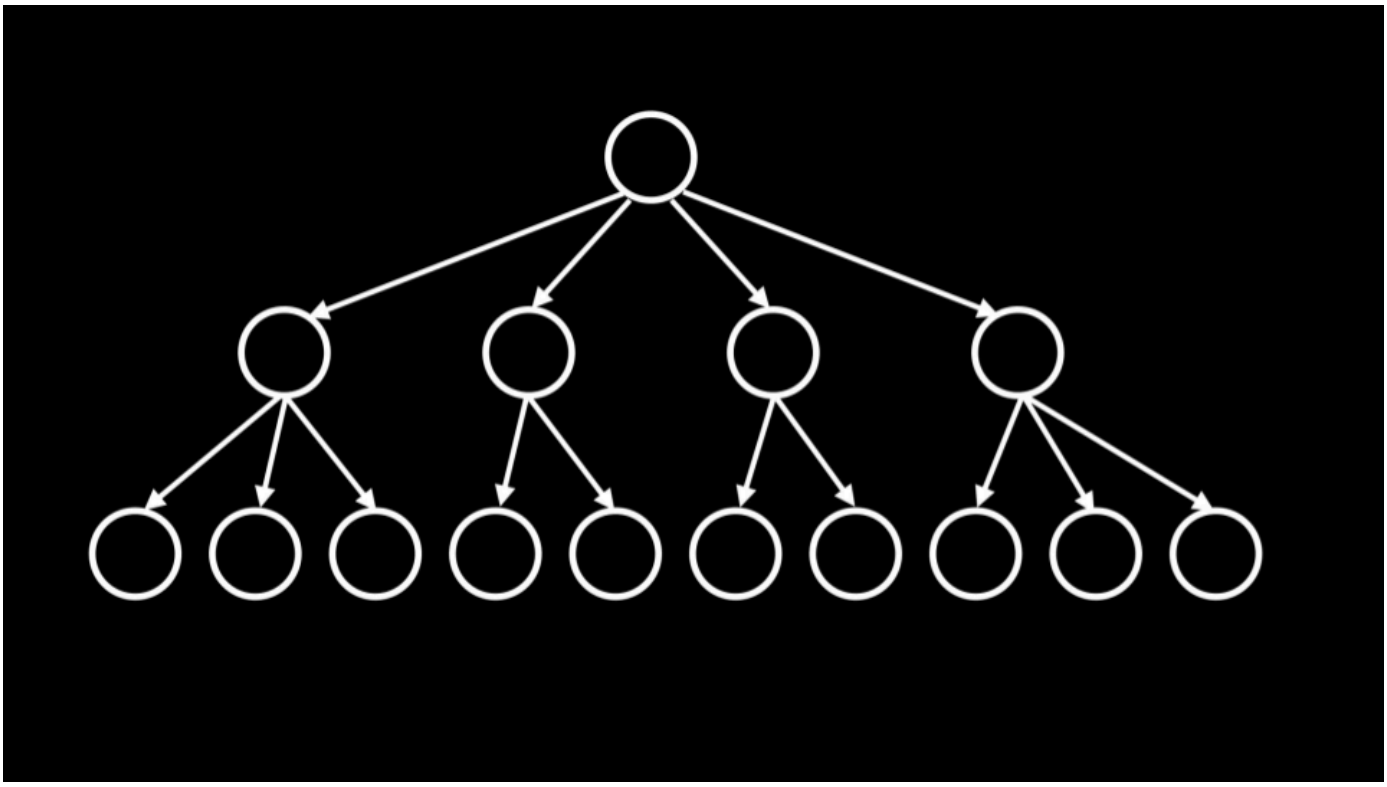
```
SELECT title FROM shows, stars, people
WHERE shows.id = stars.show_id
```

```
AND people.id = stars.person_id
AND name = 'Steve Carell';
```

- The wildcard `%` operator can be used to find all people whose names start with `Steve C` one could employ the syntax `SELECT * FROM people WHERE name LIKE 'Steve C%';`.

Indexes

- While relational databases have the ability to be faster and more robust than utilizing a `CSV` file, data can be optimized within a table using *indexes*.
- Indexes can be utilized to speed up our queries.
- We can track the speed of our queries by executing `.timer on` in `sqlite3`.
- To understand how indexes can speed up our queries, run the following: `SELECT * FROM shows WHERE title = 'The Office';` Notice the time that displays after the query executes.
- Then, we can create an index with the syntax `CREATE INDEX title_index ON shows (title);`. This tells `sqlite3` to create an index and perform some special under-the-hood optimization relating to this column `title`.
- This will create a data structure called a *B Tree*, a data structure that looks similar to a binary tree. However, unlike a binary tree, there can be more than two child nodes.



- Further, we can create indexes as follows:

```
CREATE INDEX name_index ON people (name);
CREATE INDEX person_index ON stars (person_id);
```

- Running the query and you will notice that the query runs much more quickly!

```
SELECT title FROM shows WHERE id IN
    (SELECT show_id FROM stars WHERE person_id =
        (SELECT id FROM people WHERE name = 'Steve Carell'));
```

- Unfortunately, indexing all columns would result in utilizing more storage space. Therefore, there is a tradeoff for enhanced speed.

Using SQL in Python

- To assist in working with SQL in this course, the CS50 Library can be utilized as follows in your code:

```
from cs50 import SQL
```

- Similar to previous uses of the CS50 Library, this library will assist with the complicated steps of utilizing SQL within your Python code.
- You can read more about the CS50 Library's SQL functionality in the documentation (https://cs50.readthedocs.io/libraries/cs50/python/#cs50.SQL).
- Using our new knowledge of SQL, we can now leverage Python alongside.
- Modify your code for `favorites.py` as follows:

```
# Searches database popularity of a problem

from cs50 import SQL

# Open database
db = SQL("sqlite:///favorites.db")

# Prompt user for favorite
favorite = input("Favorite: ")

# Search for title
rows = db.execute("SELECT COUNT(*) AS n FROM favorites WHERE language = ?", favor

# Get first (and only) row
row = rows[0]

# Print popularity
print(row["n"])
```

Notice that `db = SQL("sqlite:///favorites.db")` provides Python the location of the database file. Then, the line that begins with `rows` executes SQL commands utilizing `db.execute`. Indeed, this command passes the syntax within the quotation marks to the `db.execute` function. We can issue any SQL command using this syntax. Further, notice that `rows` is returned as a list of dictionaries. In this case, there is only one result, one row, returned to the rows list as a dictionary.

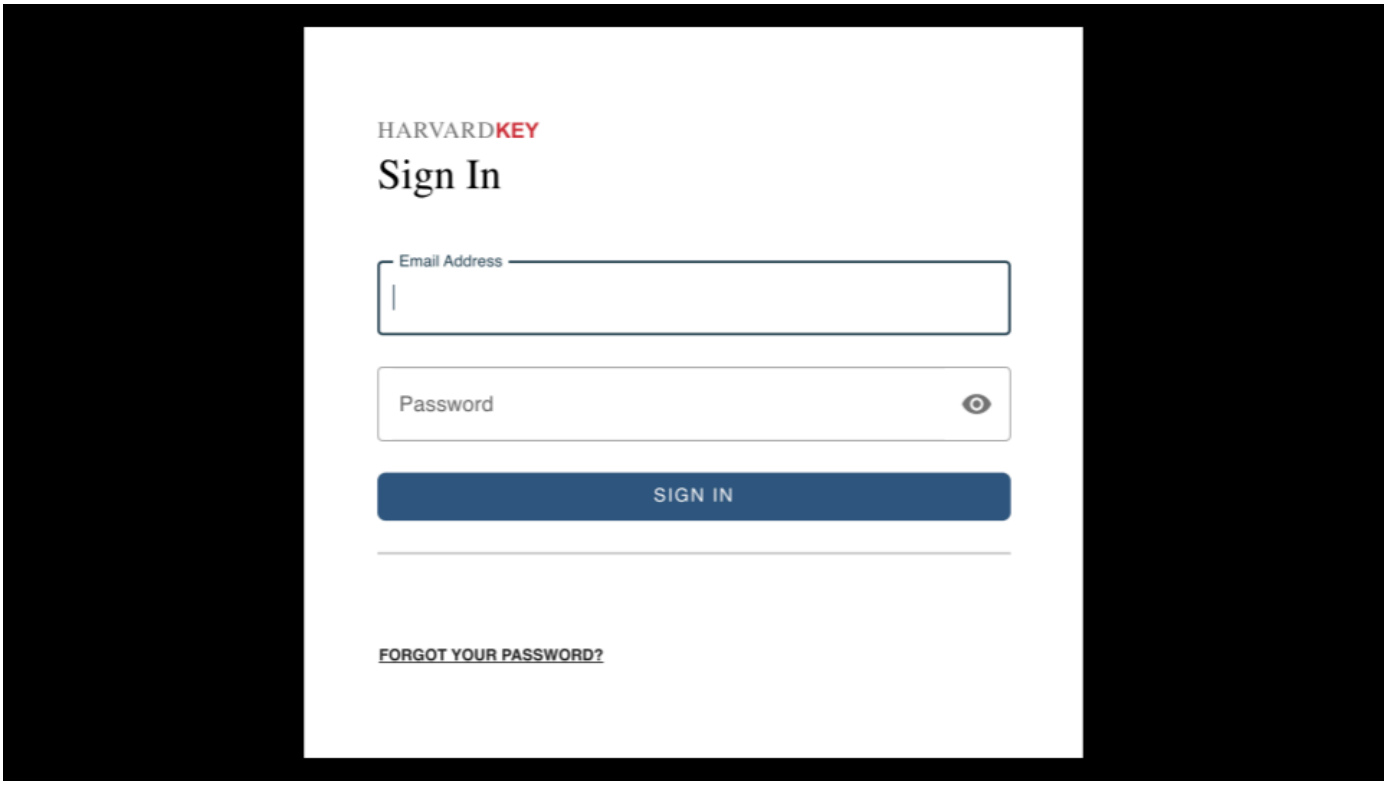
Race Conditions

- Utilization of SQL can sometimes result in some problems.
- You can imagine a case where multiple users could be accessing the same database and executing commands at the same time.
- This could result in glitches where code is interrupted by other people's actions. This could result in a loss of data.

- Built-in SQL features such as `BEGIN TRANSACTION`, `COMMIT`, and `ROLLBACK` help avoid some of these race condition problems.

SQL Injection Attacks

- Now, still considering the code above, you might be wondering what the `?` question marks do above. One of the problems that can arise in real-world applications of SQL is what is called an *injection attack*. An injection attack is where a malicious actor could input malicious SQL code.
- For example, consider a login screen as follows:



- Without the proper protections in our own code, a bad actor could run malicious code. Consider the following:

```
rows = db.execute("SELECT COUNT(*) FROM users WHERE username = ? AND password = ?
```

  Notice that because the `?` is in place, validation can be run on `favorite` before it is blindly accepted by the query.
- You never want to utilize formatted strings in queries as above or blindly trust the user's input.
- Utilizing the CS50 Library, the library will *sanitize* and remove any potentially malicious characters.

Summing Up

In this lesson, you learned more syntax related to Python. Further, you learned how to integrate this knowledge with data in the form of flat-file and relational databases. Finally, you learned about *SQL*. Specifically, we discussed…

- Flat-file databases
- Relational databases

- SQL commands such as `SELECT`, `CREATE`, `INSERT`, `DELETE`, and `UPDATE`.
- Primary and foreign keys
- `JOIN`s
- Indexes
- Using SQL in Python
- Race conditions
- SQL injection attacks

See you next time!